Do WebForms ao C# organizado

FutSimulator | Etapa 2 | Guia prático de leitura do seu projeto

> **Você já conhece a lógica. Agora vamos localizar cada responsabilidade.**
> Validar campos, cadastrar um clube e mostrar o resultado continuam sendo tarefas familiares. A diferença é que a tela, o fluxo do cadastro, as regras e o banco estão em lugares diferentes.

O que você vai conseguir acompanhar

1. Quem executa o React, o C# e o SQL.
2. O que fazem API, Application, Domain e Infrastructure.
3. A viagem completa do clique em Criar clube até os 11 jogadores salvos.
4. Onde alterar uma regra e onde colocar um breakpoint.
5. Como separar problemas de tela, HTTP, configuração e banco.

Como estudar

Deixe o VS Code aberto ao lado do PDF. Leia uma seção e encontre os arquivos citados. Os caminhos são relativos à pasta **BrasfootAPI**. Um método só fica claro quando você vê quem o chama, o que ele recebe e o que devolve.

O texto explica o código auditado desta Etapa 2. Os nomes atuais em inglês são mantidos nas citações para você encontrá-los no editor. Exemplos novos usam português brasileiro; não foram feitas renomeações nem mudanças no jogo para produzir este material.

Edição de estudo: 13/10/2026, America/Cuiaba.
Base: código local, documentação do projeto e referências oficiais ao final.

Seu roteiro de leitura

Você pode ler em ordem ou voltar diretamente a uma dúvida.

Página	Assunto	Pergunta que responde
03	WebForms x este projeto	Onde foi parar o code-behind?
04	Onde tudo executa	As camadas são servidores diferentes?
05	Mapa das pastas	O que eu abro primeiro?
06	Inicialização e DI	Quem cria as classes e chama os métodos?
07	Fluxo de criação	O que acontece depois do clique?
08	Caso de uso e DTOs	Quem valida e decide o cadastro?
09	Entidades e gerador	Como surgem os 11 jogadores?
10	EF Core e transação	Quando o banco realmente recebe os dados?
11	React e formulário	Como a tela muda sem postback?
12	Escudos e cores	Por que a cor muda sem salvar?
13	Listagem e amistoso	Como os clubes entram na partida?
14	Migrations e comandos	Atualizar o schema é iniciar a API?
15	Erros e depuração	Onde eu procuro quando quebra?
16	Exercícios guiados	Como eu aprendo mexendo com segurança?
17	Vocabulário do código	O que significam async, record e interface?
18	Testes e referências	Como verificar e continuar estudando?

Pergunta para levar a todas as páginas
Este trecho apresenta dados, decide uma regra, transporta uma mensagem ou acessa o banco? A resposta ajuda a escolher a camada correta.

Você já conhece metade disso

Use WebForms como referência, sem tentar reproduzir todo o ciclo de uma página.

No WebForms, você reconhece a página .aspx, o code-behind, eventos como **Page_Load** e o clique de um botão. Neste projeto, o formulário é React, e o servidor recebe pedidos HTTP que devolvem dados JSON. A separação por camadas também pode existir em WebForms; aqui ela está explícita em projetos C# separados.

Referência em WebForms	Onde procurar aqui
.aspx e controles visuais	CreateTeamPage.tsx e componentes React no navegador.
btnSalvar_Click no servidor	O onSubmit começa no navegador. A ação de servidor entra em POST /teams/custom.
Regras no code-behind ou em uma classe de negócio	CustomTeamService coordena o cadastro; PlayerGenerator gera o elenco.
Classe de acesso a dados	EfGameStore e AppDbContext usam EF Core para falar com MySQL.
GridView e DataBind	A tela recebe players e usa map para montar as linhas da tabela.
MasterPage / controles .ascx	AppLayout organiza a moldura; componentes como TeamBadge são reutilizados.
ViewState / IsPostBack	Não há ViewState do WebForms neste app. Estado do formulário e cache da consulta ficam no cliente.

O clique não chama uma classe C# diretamente

```
// Exemplo didático de um evento WebForms; não é arquivo do projeto.
protected void SalvarClubeAoClicar(object origem, EventArgs argumentos)
{
    string nomeDoClube = txtNome.Text.Trim();
    // Validar, gravar e atualizar a tela poderiam ocorrer aqui.
}
```

Aqui essas tarefas continuam existindo, mas o JavaScript envia uma mensagem ao servidor. O endpoint escolhe o serviço C#; o resultado volta pela rede, e o React atualiza a apresentação. Mudar uma cor da prévia não precisa de uma viagem ao servidor.

Referências conceituais: ciclo de página WebForms [1] e renderização React [2], página 18.

Onde cada coisa executa

Processos em execução e dependências entre projetos são mapas diferentes.



Em desenvolvimento, há ainda o **Vite na porta 5173**: um processo Node que entrega os arquivos e recompila o frontend. A tela React deste projeto é executada no navegador, não no Vite nem no servidor C#. O Workbench é um cliente do MySQL; abrir o Workbench não substitui iniciar o serviço do banco.

Como os projetos C# se conhecem

```
Api -> Application -> Domain
Api -> Infrastructure -> Application -> Domain

No cadastro, Application chama IGameStore.
Em execução, a DI entrega EfGameStore para essa interface.
```

> **Uma camada não é um microsserviço**
> Você inicia a API uma vez. Ela carrega as bibliotecas das outras camadas. Não precisa abrir quatro terminais para quatro projetos do backend.

Dependência em código x chamada em execução

Application conhece o contrato IGameStore, não a classe de EF Core. Infrastructure implementa esse contrato. Isso permite ao cadastro pedir uma gravação sem importar DbContext. O fluxo de execução cruza a interface, mas a referência entre projetos continua apontando da infraestrutura para a aplicação.

Mapa do projeto

Abra a solução para ver responsabilidades; abra os arquivos para ver o fluxo.

Pasta / projeto	O que faz	Arquivos de entrada
backend/ MiniBrasfoot.Api	Recebe HTTP, registra dependências, configura MySQL, CORS e erros.	Program.cs Endpoints/TeamEndpoints.cs Middleware/ApiExceptionHandler.cs
backend/ MiniBrasfoot.Application	Coordena os casos de uso, valida dados de entrada e define contratos.	Services/CustomTeamService.cs DTOs/Contracts.cs Interfaces/IGameStore.cs
backend/ MiniBrasfoot.Domain	Representa clubes/jogadores e regras independentes de HTTP/EF.	Entities/Team.cs Entities/Players.cs Rules/PlayerGenerator.cs Rules/MatchSimulator.cs
backend/ MiniBrasfoot.Infrastructure	Traduz a gravação para EF/MySQL e descreve o schema.	Persistence/EfGameStore.cs Persistence/AppDbContext.cs Configurations/ Migrations/
backend/ MiniBrasfoot.Tests	Executa verificações sem gravar no MySQL do usuário.	CustomTeamTests.cs PlayerGeneratorTests.cs CustomTeamPersistenceTests.cs
frontend/src	Apresentação, formulário, HTTP e estado no navegador.	main.tsx app/App.tsx routes/CreateTeamPage.tsx features/teams/

Arquivos que dão suporte

BrasfootAPI.sln reúne os projetos; um **.csproj** declara bibliotecas e referências. **Directory.Build.props** define net10.0 em comum; **global.json** orienta a escolha do SDK. **frontend/package.json** descreve scripts e dependências do frontend.

docs/ explica decisões. **tools/DatabaseProbe** faz consultas de diagnóstico. **legacy/frontend** guarda a versão antiga. **scripts/** contém utilitários: eles só rodam quando alguém os executa. **output/pdf/** guarda materiais como este guia.

Quem liga o backend?

Program.cs monta a aplicação; a injeção de dependências monta os objetos.

Ao executar **dotnet run** no projeto Api, Program.cs configura o host. Ele registra serviços, define como conectar ao MySQL, monta o tratamento HTTP, mapeia as rotas e inicia a escuta. Não existe um Page_Load de cada tela React no backend.

Trecho real de Program.cs

```
builder.Services.AddScoped<IGameStore, EfGameStore>();
builder.Services.AddSingleton<PlayerGenerator>();
builder.Services.AddScoped<CustomTeamService>();

var app = builder.Build();
app.UseExceptionHandler();
app.UseCors();
app.MapTeamEndpoints();
app.Run();
```

Registro	Tradução para o dia a dia
AddScoped<IGameStore, EfGameStore>	Quando alguém pedir IGameStore, entregue EfGameStore. No fluxo HTTP, a instância é compartilhada dentro do escopo daquela requisição.
AddScoped<CustomTeamService>	Crie o coordenador do cadastro e forneça as dependências registradas que ele pede.
AddSingleton<PlayerGenerator>	Compartilhe uma instância do gerador na aplicação. Ele não guarda um elenco mutável entre chamadas.

O construtor que parece uma declaração curta

```
public sealed partial class CustomTeamService(
    IGameStore store, PlayerGenerator generator)
{
    // A classe usa as dependências recebidas pelo construtor.
}
```

É um **construtor primário** de C#. store e generator são dependências recebidas. O endpoint pede CustomTeamService; a DI resolve a árvore necessária, inclusive AppDbContext. Esse DbContext é scoped por padrão no registro AddDbContext. Não é criado um por cada jogador.

Lifetimes: scoped é por escopo HTTP; singleton é compartilhado. Fonte [3], página 18.

A viagem do clique ao banco

Leia de cima para baixo. A resposta percorre o caminho de volta.

1. **CreateTeamPage.tsx - clique em Criar clube**
   handleSubmit valida o formulário com Zod, no navegador.
2. **use-create-team.ts**
   mutation.mutate inicia o envio e controla loading, sucesso e erro.
3. **features/teams/api.ts + services/http.ts**
   Axios envia POST /teams/custom com dados JSON para a API.
4. **TeamEndpoints.cs - servidor C#**
   Recebe CustomTeamInput e chama CustomTeamService.Create.
5. **CustomTeamService.cs**
   Valida, pede o elenco, calcula a força e monta o clube.
6. **PlayerGenerator.cs**
   Gera exatamente 11 jogadores em memória, sem banco.
7. **IGameStore -> EfGameStore**
   A implementação inicia uma transação e faz duas gravações EF.
8. **MySQL -> DTO -> HTTP 201**
   Commit confirma; o servidor devolve clube e jogadores com IDs.
9. **React exibe o clube e os 11 jogadores**
   O cache de ['teams'] é invalidado para atualizar a listagem.

Dois pedidos diferentes
GET /teams apenas lista. POST /teams/custom cria clube e elenco. Abrir a página de criação ou trocar a cor da prévia não salva um clube.

Application: o cadastro

CustomTeamService.Create é o coordenador do caso de uso.

Um **caso de uso** é uma ação completa do sistema: neste caso, criar um clube com identidade e elenco. Ele não desenha controles nem escreve SQL. Ele organiza as decisões do cadastro.

Sequência real, em trechos curtos

```
Validate(input);
var players = generator.Generate();

// Dentro da montagem de Team:
Strength = (int)players.Average(player => player.Skill)

await store.CreateTeamWithPlayers(team, players, ct);
// Depois: mapear o clube e cada jogador para DTOs de resposta.
```

Decisão	Onde está e por que fica no servidor
Nome e país	Validate rejeita branco e mais de 100 caracteres após trim. A interface pode avisar antes, mas pedidos diretos à API também precisam ser validados.
Template e cores	Só os três identificadores previstos e cores #RRGGBB. ToUpperInvariant normaliza as cores antes de persistir.
Força e IsCustom	São definidos pelo caso de uso. O cliente não escolhe uma força arbitrária para o cadastro personalizado.

DTO não é a entidade do banco

CustomTeamInput é o contrato de entrada: nome, país, template e duas cores. **CustomTeamDto** é a resposta: TeamDto + lista de PlayerDto. Estão em DTOs/Contracts.cs. São records para transportar dados; não fazem a gravação por conta própria.

```
{
  "name": "Aurora FC", "country": "Brasil",
  "badgeTemplate": "shield-round",
  "primaryColor": "#165DFF", "secondaryColor": "#FFD700"
}
```

ASP.NET Core converte JSON em DTO de entrada e serializa DTO de resposta em JSON. Na rede, o contrato usa camelCase. Name é a propriedade C#; name é a chave JSON. A resposta 201 inclui Location: /teams/{id}.

Domain: clube e jogadores

Objetos do jogo e regras que podem funcionar sem HTTP ou MySQL.

Team.cs representa um clube: Id, Name, Country, Strength e os quatro campos de identidade. **Players.cs** contém a classe Player: Id, Name, Age, Skill, TeamId e Position. Uma entidade pode existir só na memória; instanciá-la não a grava no banco.

Função na formação	Quantidade	Código persistido
Goleiro	1	GK
Zagueiros	2	DEF
Laterais	2	DEF
Volantes	2	MEI
Meias	2	MEI
Atacantes	2	ATT

O gerador usa 11 slots nessa ordem. O modelo atual agrupa zagueiros/laterais como DEF e volantes/meias como MEI; não existe um campo separado de subfunção. A regra de goleiro continua usando GK exatamente em maiúsculas.

Por que os nomes não se repetem?

PlayerGenerator combina 12 nomes com 12 sobrenomes, formando 144 opções. A cada jogador, sorteia um índice e remove o nome escolhido da lista de opções. Não tenta adivinhar um nome novo repetidamente; a escolha sem reposição garante unicidade dentro daquele elenco.

```
Age = random.Next(18, 33)   // Valores possíveis: 18 até 32.
Skill = random.Next(50, 81) // Valores possíveis: 50 até 80.
```

O limite superior de Random.Next é exclusivo. Generate aceita Random opcional: Random.Shared em produção; new Random(42) num teste reproduz a sequência, no mesmo ambiente de execução. Os testes também podem fornecer outra implementação de Random.

> **Força inicial**
> Habilidades 60, 65, 70, 75, 68, 66, 69, 72, 64, 67, 70 somam 746. 746 / 11 = 67,818...; a conversão para int resulta em Strength = 67. Não é arredondamento para 68.

Infrastructure: gravar de verdade

EfGameStore liga a intenção do cadastro ao EF Core e ao MySQL.

IGameStore declara as operações de persistência. **EfGameStore** implementa essas operações. **AppDbContext** acompanha entidades e oferece DbSet<Team> Teams e DbSet<Player> Players. Pomelo é o provider que permite ao EF Core trabalhar com MySQL.

1. **BeginTransactionAsync**
   Abre uma transação para abranger clube e elenco.
2. **db.Teams.Add(team)**
   Coloca o clube no ChangeTracker. Add ainda não executa o INSERT.
3. **Primeiro SaveChangesAsync**
   Executa a gravação; o MySQL gera o Id do clube.
4. **Preencher TeamId e AddRange(players)**
   Os 11 jogadores recebem o Id e são acompanhados pelo EF.
5. **Segundo SaveChangesAsync + CommitAsync**
   Grava o elenco e confirma o conjunto inteiro.

Se falhar o sexto jogador?

O catch faz RollbackAsync, limpa o ChangeTracker e relança a exceção. Mesmo que o clube já tenha sido inserido dentro da transação, a gravação não fica confirmada sem o elenco. Isso depende de tabelas transacionais, como InnoDB.

> **Duas gravações; uma confirmação**
> Uma transação automática por SaveChanges não uniria sozinha as duas chamadas. A transação explícita envolve ambas. A documentação EF Core [4] explica esse comportamento.

Um rollback não promete devolver os números de auto_increment já utilizados. Um intervalo na sequência de IDs não significa que existe um clube parcialmente cadastrado.

React: o lado da tela

Os componentes organizam apresentação e eventos, não as regras do elenco.

Arquivo / ferramenta	O que faz durante a criação
main.tsx + app/App.tsx	main monta React no HTML. App instala QueryClient/rotas e seleciona CreateTeamPage em /clubs/new.
CreateTeamPage.tsx	Monta campos, prévia e botão. Após sucesso, usa os dados retornados para mostrar clube e jogadores.
React Hook Form + schema.ts	useForm registra campos; Zod valida formato para feedback. handleSubmit só chama a mutation se o formulário for válido.
Controller + useWatch	Controller conecta componentes controlados ao formulário. useWatch acompanha valores para atualizar a prévia local.
use-create-team.ts	TanStack Query usa useMutation para controlar o envio. No sucesso, invalida ['teams']; a próxima consulta pode buscar a lista atualizada.
features/teams/api.ts	createCustomTeam usa o cliente Axios. A URL da operação fica aqui, não espalhada pelos controles.
services/http.ts	Define baseURL/timeout. getHttpError converte falhas em texto para a tela.
Button, Card, AppLayout e globals.css	Organizam aparência, layout e tema. Não geram jogadores nem calculam força.

Estados que substituem o 'recarregar tudo'

isPending: botão e campos ficam desabilitados durante o envio. **isError**: a mensagem da operação aparece. **isSuccess**: mutation.data contém o resultado, e a função do componente descreve outra apresentação.

React pode executar a função do componente novamente para calcular a interface. Isso não significa novo cadastro: o POST está no evento de envio, não solto no corpo da função. Estado do cliente também não equivale a dados persistidos no MySQL. Referência [2].

Escudos: a prévia é local

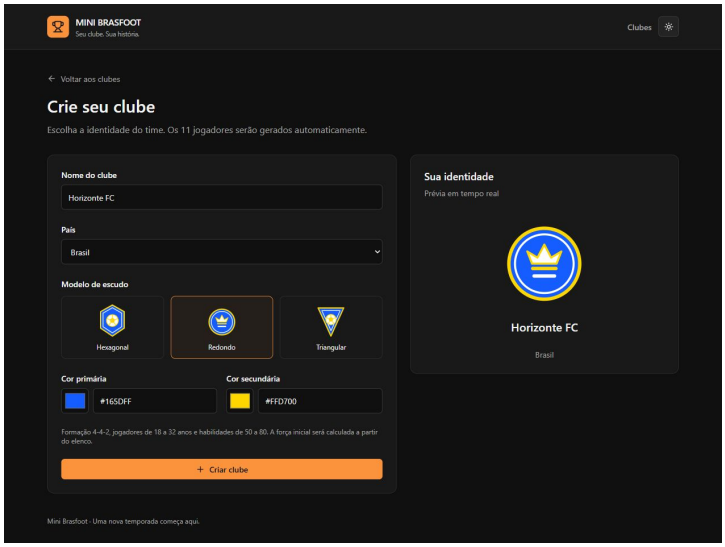
O banco guarda escolhas; os desenhos continuam sendo arquivos SVG do frontend.

Componente	Responsabilidade
BadgeSelector.tsx	Mostra três radios com os modelos e informa qual foi escolhido.
ColorPicker.tsx	Oferece seletor visual e campo hexadecimal, com feedback de formato.
BadgePreview.tsx	Apresenta o escudo maior e o nome digitado.
TeamBadge.tsx	Escolhe o componente SVG e aplica duas variáveis CSS por instância.

O mesmo desenho; valores diferentes

```
// Trecho real do estilo montado em TeamBadge:
'--primary-color': validColor(primaryColor, '#F97316'),
'--secondary-color': validColor(secondaryColor, '#17212B'),
```

Os SVGs usam fill/stroke com var(--primary-color, ...) e var(--secondary-color, ...). SVGR transforma o import com **?react** em um componente inline. No Vite, svgo: false preserva os atributos do desenho. Um img apontando para um SVG não receberia essas variáveis locais da mesma maneira.



Evidência da interface na QA da Etapa 2, com API simulada. A coroa continua branca. A captura não representa dados atuais do seu banco.

> **Alterar a prévia não altera o banco**
> useWatch atualiza os valores locais. Só Criar clube envia template e cores. No MySQL não há markup SVG. Clube sem template usa iniciais neutras.

Listar e jogar: outros fluxos

A mesma arquitetura atende ações diferentes, cada uma com sua própria rota.

Listagem de clubes

```
TeamsPage.tsx -> useTeams() -> getTeams()
GET /teams -> TeamEndpoints -> TeamService.List
IGameStore.ListTeams -> EfGameStore -> MySQL
TeamDto[] -> cards com TeamBadge
```

use-teams.ts usa a chave de cache **['teams']**. Após criação, a mutation invalida essa chave. TeamService.Map inclui identidade e força no DTO. Em EfGameStore, AsNoTracking é usado na listagem: esses objetos lidos não entram no acompanhamento de alterações daquela consulta.

Amistoso

```
HomePage.tsx -> features/matches/api.ts
POST /match?team1Id=X&team2Id=Y
MatchEndpoints -> MatchService.Simulate
Consultar clubes/elencos -> MatchSimulator.Simulate
MatchDto -> placar e autores dos gols
```

MatchService valida os clubes e busca os elencos. MatchSimulator recebe objetos e listas para calcular o resultado sem acessar o banco. O motor já existia; sua matemática não foi alterada pela Etapa 2.

Regra atual	Como interpretar
Força efetiva	Strength do clube + média truncada de Skill de todo o elenco, incluindo o goleiro.
Goleiros e gols	Position == GK fica fora do sorteio de autores de gols.
Persistência de partida	O resultado é devolvido ao cliente, mas histórico de partidas ainda não tem tabela neste projeto.
Escudos no placar	O frontend recupera a identidade dos clubes selecionados e usa TeamBadge. MatchDto continua com o contrato anterior.

Etapa futura
Competition, chaves, pênaltis e reprodução de 30 segundos foram documentados, não implementados. Eles não estão escondidos em alguma classe da Etapa 2.

Schema, migrations e comandos

A classe C# descreve o modelo; a migration altera o banco para acompanhar esse modelo.

Arquivo	O que representa
TeamConfiguration.cs	Mapeamento EF: IsCustom tem default false; template até 32, cores até 7; identidade pode faltar em clubes anteriores.
PlayerConfiguration.cs	Relacionamento Players.TeamId -> Teams.Id, com exclusão restrita.
AddCustomTeamBadges.cs	Up acrescenta quatro colunas. Down remove essas colunas; não é um botão seguro de desfazer dados personalizados.
.Designer.cs e ModelSnapshot	Metadados gerados e retrato do modelo usado pelo EF para comparar mudanças.
AppDbContextFactory.cs	Cria o DbContext para comandos dotnet ef. Lê variáveis de ambiente ou fallback local; não é o caminho normal das requisições HTTP.

São operações diferentes

```
# Iniciar a API:
dotnet run --project backend/MiniBrasfoot.Api --launch-profile http

# Instalar o frontend a partir do lockfile e iniciar o Vite:
npm.cmd --prefix frontend ci
npm.cmd --prefix frontend run dev
```

npm.cmd evita o bloqueio de npm.ps1 no seu PowerShell. ci reinstala dependências conforme o lockfile; run dev inicia a interface. dotnet build compila, mas não aplica migration. dotnet ef migrations add gera arquivos; database update aplica as migrations pendentes.

```
# Comando de schema: somente após backup e revisão das pendentes.
dotnet ef database update `
  --project backend/MiniBrasfoot.Infrastructure `
  --startup-project backend/MiniBrasfoot.Api
```

O que sabemos nesta conversa
Você informou que fez backup. Nas imagens anteriores só InitialMySql estava aplicada e faltavam as quatro colunas. A conclusão da atualização não foi confirmada. Confira __EFMigrationsHistory e SHOW COLUMNS FROM Teams no seu banco.

A API não aplica migrations automaticamente. DDL MySQL pode fazer commit implícito; a transação da página 10 protege os INSERTs do cadastro, não promete desfazer ALTER TABLE.

Quando der erro, siga o pedido

O local da mensagem na tela não é necessariamente o local da causa.

Sintoma	Onde conferir primeiro
npm.ps1 bloqueado	Terminal PowerShell: usar npm.cmd. Não é um erro de React nem de banco.
Tela não abre	Terminal do Vite e URL localhost:5173. Verifique se o processo está ativo e qual porta anunciou.
Não conectou à API	Backend na porta 5063 e VITE_API_URL em .env.local. Após mudar configuração VITE, reinicie o Vite.
HTTP 400	Dados enviados e validação no serviço. Campos inválidos no formulário também podem bloquear o POST antes da rede.
HTTP 409	Relacionamento/conflito: exclusão de clube com jogadores ou falha de FK. Não apagar dados para contornar.
HTTP 503	Servidor/credenciais/schema do MySQL. 'Banco de dados indisponível' é genérico; não prova sozinho que o serviço está parado.
Prévia não muda	useWatch, props de TeamBadge, variáveis CSS e SVG inline. Não começar olhando o SQL.

Quem transforma exceções em mensagens?

ApiExceptionHandler.cs mapeia exceções para códigos HTTP e ProblemDetails. InputRules/CustomTeamService usam UseCaseException para erros esperados. No frontend, getHttpError lê title da resposta. Logs mostram tipo/traceId; não imprimem a senha de conexão.

Breakpoints no VS Code

Inicie o backend pelo depurador C# ou conecte o depurador ao processo já iniciado. Só desenhar um ponto vermelho e executar dotnet run num terminal não garante depuração. Coloque pontos no endpoint, em Create, em Generate e antes de cada SaveChangesAsync.

Ferramentas do navegador
F12 > Network mostra método, URL, payload e status do POST. Se nenhum POST apareceu, comece pelo formulário. Se ele chegou e retornou erro, siga para o C#. Evite compartilhar senhas ou strings de conexão nas capturas.

Aprenda fazendo um percurso

Cinco exercícios curtos para conhecer o sistema com o editor aberto.

1. Leia o cadastro sem executar

Abra CreateTeamPage.tsx e encontre handleSubmit. Siga useCreateTeam até createCustomTeam. Depois abra TeamEndpoints.cs e localize /custom. Em cada ponto, escreva: 'recebe X, chama Y, devolve Z'. Você terá o mapa da página 7 feito por você.

2. Observe a memória antes da gravação

Em ambiente de desenvolvimento com o depurador, pare após generator.Generate(). Inspecione players.Count, Position, Age e Skill. Nesse momento não houve INSERT de clube/jogador pelo cadastro. Os IDs novos ainda serão obtidos na persistência. Se concluir o cadastro, será criado um clube real: use uma cópia do banco para experimentar.

3. Acompanhe os IDs

Pare em EfGameStore antes e depois do primeiro SaveChangesAsync. Compare team.Id. Depois acompanhe TeamId em cada jogador. Entenda por que o clube precisa do ID antes de o elenco ser salvo nesta implementação.

4. Teste uma regra sem abrir o site

```
// Exemplo novo em português, usando a classe existente.
var geradorDeJogadores = new PlayerGenerator();
var jogadores = geradorDeJogadores.Generate(new Random(42));
int quantidadeDeJogadores = jogadores.Count; // Deve ser 11.
int forcaInicial = (int)jogadores.Average(jogador => jogador.Skill);
```

Esse trecho é um exemplo para um teste ou experimento separado, não uma alteração já aplicada. Para mudar os limites de habilidade no produto, a regra começa no gerador, mas testes e texto da tela também precisam acompanhar a mudança.

5. Separe cor visual de dado persistido

Abra /clubs/new, altere as cores e acompanhe Network. A prévia muda sem POST. Observe TeamBadge recebendo os valores. O salvamento só acontece quando o formulário é enviado.

> **Onde colocar uma mudança?**
> Texto e layout: React. Regra do cadastro: Application. Geração e simulação: Domain. Mapeamento/SQL: Infrastructure. Contrato HTTP e DI: Api. Sempre acompanhe o resultado com um teste apropriado.

Vocabulário para ler o C#

Traduções práticas dos termos que aparecem nos arquivos atuais.

Termo	Como ler neste projeto
class / entidade	Um tipo com propriedades/comportamento. Team representa um clube; não é a tabela em execução.
interface	Um contrato de operações. IGameStore permite trocar a implementação sem mudar o caso de uso.
record / DTO	Um tipo adequado para dados de entrada/saída. Não grava nada sozinho.
sealed / partial	sealed impede herdar a classe. partial permite partes da mesma classe; GeneratedRegex gera outra parte durante a compilação.
async / await / Task<T>	O método pode aguardar trabalho assíncrono e produzir T depois. await continua após o resultado; não cria automaticamente uma thread para cada chamada.
CancellationToken ct	Sinaliza cancelamento e é repassado às operações. Não substitui Commit/Rollback.
IReadOnlyList<Player>	Uma lista que esse contrato não oferece para adicionar/remover. Os objetos Player ainda podem ter propriedades alteradas, como TeamId.
LINQ: Select / Average	Select transforma cada item; Average calcula a média. Usados para montar DTOs e calcular força.
DbSet / ChangeTracker	DbSet é a entrada EF para aquele tipo; ChangeTracker acompanha estados antes de SaveChanges.
namespace / using	namespace organiza nomes de tipos. using permite referenciar tipos ou recursos. Não significa executar aquele arquivo.
JSON / props / hook	JSON transporta dados; props são entradas do componente; hook conecta comportamento/estado ao React.

Leia esta linha em voz alta

```
await store.CreateTeamWithPlayers(team, players, ct);
```

'Peça ao armazenamento para criar o clube com seus jogadores e aguarde a conclusão, propagando o cancelamento.' A interface descreve o pedido; a implementação EfGameStore faz a transação. A linha seguinte só monta a resposta depois que esse pedido termina com sucesso.

Conceito async/await: referência [5], página 18.

Verifique e continue estudando

Os testes permitem entender o comportamento sem depender de cliques manuais.

Teste	O que ensina
PlayerGeneratorTests.cs	Quantidade, códigos de posição, nomes únicos, limites, semente e compatibilidade com o simulador.
CustomTeamTests.cs	Caso de uso e HTTP: validação, força, 201/Location, retorno do elenco e contratos antigos.
CustomTeamPersistenceTests.cs	Transação real em SQLite em memória: sucesso e rollback preservando dados anteriores. Não comprova escrita no MySQL real.
FakeGameStore.cs + ApiTests.cs	Fake atende o contrato sem banco. ApiTests cobre rotas históricas, CORS e erros.
TeamBadge.test.tsx + schema.test.ts	SVGs originais/cores/fallback e validação do formulário. MatchSimulatorTests.cs protege as regras da simulação.

```
# Compilar e testar backend; rodar os comandos na raiz.
dotnet build BrasfootAPI.sln -m:1 /nr:false
dotnet test backend/MiniBrasfoot.Tests --no-build

# Frontend; não precisa do MySQL para estes testes unitários.
npm.cmd --prefix frontend run test
npm.cmd --prefix frontend run lint
```

A entrega da Etapa 2 registrou 58 testes .NET e 16 testes frontend aprovados. Esses são resultados daquela execução, não uma nova execução feita para este PDF. Em docs/VALIDATION.md estão as limitações e evidências.

Referências oficiais para aprofundar

[1] Microsoft: ciclo de vida de páginas WebForms
[2] React: renderização e atualização do DOM
[3] Microsoft: ciclos de vida dos serviços na DI
[4] Microsoft: transações no EF Core
[5] Microsoft: cenários assíncronos em C#

A base deste guia é o seu código
Program.cs, serviços, entidades, persistência, componentes e testes foram lidos no projeto local. As referências complementam os conceitos; os fluxos, caminhos e regras descritos são os da implementação auditada.